Document number: P1358R0
Date: 2019-02-22
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2017
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

# Core Language Working Group "ready" Issues for the February, 2019 (Kona) meeting

References in this document reflect the section and paragraph numbering of document WG21 N4778.

---

### 2256. Lifetime of trivially-destructible objects

**Section:** 6.6.3 [basic.life] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-30

According to 6.4 [basic.lookup] bullet 1.4, the following example has defined behavior because the lifetime of `n` extends until its storage is released, which is after `a`'s destructor runs:

```
void f() {
  struct A { int *p; ~A() { *p = 0; } } a;
  int n;
  a.p = &n;
}
```

It would be more consistent if the end of the lifetime of all objects, regardless of whether they have a non-trivial destructor, were treated the same.

**Notes from the March, 2018 meeting:**

CWG agreed with the suggested direction.

**Proposed resolution (November, 2018):**

1. Change 6.6.3 [basic.life] paragraph 1 as follows:

> The *lifetime* of an object or reference is a runtime property of the object or reference. ~~An object is said to have *non-vacuous initialization* if it is of a class or array type and it or one of its subobjects is initialized by a constructor other than a trivial default constructor. [*Note:* : Initialization by a trivial copy/move constructor is non-vacuous initialization. — *end note*]~~ **A variable is said to have *vacuous initialization* if it is default-initialized and, if it is of class type or (possibly multi-dimensional) array thereof, that class type has a trivial default constructor.** The lifetime of an object of type `T` begins when:
>
> - storage with the proper alignment and size for type `T` is obtained, and
> - ~~if the object has non-vacuous initialization,~~ its initialization **(if any)** is complete **(including vacuous initialization) (9.3 [dcl.init]**,
>
> except that if the object is a union member or subobject thereof, its lifetime only begins if that union member is the initialized member in the union (9.3.1 [dcl.init.aggr], 10.9.2 [class.base.init]), or as described in 10.4 [class.union]. The lifetime of an object *o* of type `T` ends when:
>
> - **if `T` is a non-class type, the object is destroyed, or**
> - if `T` is a class type ~~with a non-trivial destructor (10.3.6 [class.dtor])~~, the destructor call starts, or

- the storage which the object occupies is released, or is reused by an object that is not nested within *o* (6.6.2 [intro.object]).

2. Change 6.8.3.4 [basic.start.term] paragraphs 1 and 2 as follows:

~~Destructors (10.3.6 [class.dtor]) for initialized~~ **Constructed** objects (~~that is, objects whose lifetime (6.6.3 [basic.life]) has begun~~ **9.3 [dcl.init]**) with static storage duration~~,~~ **are destroyed** and functions registered with `std::atexit`~~,~~ are called as part of a call to `std::exit` (16.5 [support.start.term]). The call to `std::exit` is sequenced before the ~~invocations of the destructors~~ **destructions** and the registered functions. [*Note:* Returning from `main` invokes `std::exit` (6.8.3.1 [basic.start.main]). —*end note*]

~~Destructors for initialized~~ **Constructed** objects with thread storage duration within a given thread are ~~called~~ **destroyed** as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The ~~completions of the destructors for~~ **destruction of** all ~~initialized~~ **constructed** objects with thread storage duration within that thread strongly happen**s** before ~~the initiation of the destructors of~~ **destroying** any object with static storage duration.

3. Change 8.6 [stmt.jump] paragraph 2 as follows:

...[*Note:* However, the program can be terminated (by calling std::exit() or std::abort() (16.5 [support.start.term]), for example) without destroying ~~class~~ objects with automatic storage duration. —*end note*]

4. Change 8.7 [stmt.dcl] paragraph 2 as follows:

It is possible to transfer into a block, but not in a way that bypasses declarations with initialization (including ones in *condition*s and *init-statement*s). A program that jumps[92] from a point where a variable with automatic storage duration is not in scope to a point where it is in scope is ill-formed unless the variable has ~~scalar type, class type with a trivial default constructor and a trivial destructor, a cv-qualified version of one of these types, or an array of one of the preceding types and is declared without an *initializer*~~ **vacuous initialization** (9.3 [dcl.init]). **In such a case, the variables with vacuous initialization are constructed in the order of their declaration.** [*Example:*...

5. Change 8.7 [stmt.dcl] paragraph 5 as follows:

~~The destructor for a~~ **A** block-scope object with static or thread storage duration will be ~~executed~~ **destroyed** if and only if it was constructed. [*Note:* 6.8.3.4 [basic.start.term] describes the order in which block-scope objects with static and thread storage duration are destroyed. —*end note*]

6. Change 9.3 [dcl.init] paragraph 21 as follows:

An object whose initialization has completed is deemed to be constructed, even if **the object is of non-class type or** no constructor of the object's class is invoked for the initialization. [*Note:* Such an object might have been value-initialized or initialized by aggregate initialization (9.3.1 [dcl.init.aggr]) or by an inherited constructor (10.9.3 [class.inhctor.init]). —*end note*] **Destroying an object of class type invokes the destructor of the class. Destroying a scalar type has no effect other than ending the lifetime of the object (6.6.3 [basic.life]). Destroying an array destroys each element in reverse subscript order.**

7. Change 10.3.6 [class.dtor] paragraph 2 as follows:

~~A destructor is used to destroy objects of its class type.~~ The address of a destructor...

8. Change 13.2 [except.ctor] paragraphs 1 and 2 as follows:

As control passes from the point where an exception is thrown to a handler, ~~destructors are invoked~~ **objects with automatic storage duration are destroyed** by a process, specified in this subclause, called *stack unwinding*.

~~The destructor is invoked for each automatic object of class type~~ **Each object with automatic storage duration is destroyed if it has been** constructed, but not yet destroyed, since the try block was entered. If an exception is thrown during the destruction of temporaries or local variables for a return statement (8.6.3 [stmt.return]), the destructor for the returned object (if any) is also invoked. The objects are destroyed in the reverse order of the completion of their construction. [*Example:*...

### 2267. Copy-initialization of temporary in reference direct-initialization

**Section:** 9.3.3 [dcl.init.ref] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-05-25

Consider the following example:

```
struct A {} a;
struct B { explicit B(const A&); };

struct D { D(); };
struct C { explicit operator D(); } c;

B b1(a);            // #1, ok
const B &b2{a};     // #2. ok
const B &b3(a);     // #3, error

D d1(c);            // ok
const D &d2{c};     // ok
const D &d3(c);     // #6, ok
```

The disparity between #3 and #6 is suprising, as is the difference from #1 and #2. The reason for this difference is in 9.3.4 [dcl.init.list] bullet 3.10:

> Otherwise, if `T` is a reference type, a prvalue of the type referenced by `T` is generated. The prvalue initializes its result object by copy-list-initialization or direct-list-initialization, depending on the kind of initialization for the reference. The prvalue is then used to direct-initialize the reference.

(reflecting the resolution of issue 1494).

**Notes from the March, 2018 meeting:**

CWG felt that initialization of the temporary should always be copy initialization, regardless of whether the top-level initialization is copy or direct initialization. This would make #2, #3, #5, and #6 all ill-formed.

**Proposed resolution (November, 2018):**

1. Change 9.3.4 [dcl.init.list] bullet 3.10 as follows:

> Otherwise, if `T` is a reference type, a prvalue of the type referenced by `T` is generated. The prvalue initializes its result object by copy-list-initialization ~~or direct-list-initialization, depending on the kind of initialization for the reference~~. The prvalue is then used to direct-initialize the reference. [Note: As usual, the binding will fail and the program is ill-formed if the reference type is an lvalue reference to a non-const type. —end note]

2. Add the following to the example in 9.3.4 [dcl.init.list] bullet 3.10:

```
struct A { } a;
struct B { explicit B(const A&); };
const B &b2(a);   // error: cannot copy-initialize B temporary from A
```

3. Change 11.3.1.6 [over.match.ref] bullet 1.1 as follows:

> ...For direct-initialization, those explicit conversion functions that are not hidden within `S` and yield type "lvalue reference to *cv2* `T2`" **(when initializing an lvalue reference or an rvalue reference to function)** ~~or "*cv2* `T2`"~~ or "rvalue reference to *cv2* `T2`" **(when initializing an rvalue reference or an lvalue reference to function)**, ~~respectively,~~ where `T2` is the same type as `T` or can be converted to type `T` with a qualification conversion (7.3.5 [conv.qual]), are also candidate functions.

### 2278. Copy elision in constant expressions reconsidered

**Section:** 7.7 [expr.const] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-06-27

The resolution of issue 2022 does not work, as it is mathematically impossible to guarantee the named return value optimization in all cases. For example:

```
struct B { B *self = this; };
extern const B b;
constexpr B f() {
  B b;
  if (&b == &::b) return B();
  else return b;
}
constexpr B b = f(); // is b.self == &b?
```

Here an implementation is required to perform the optimization if and only if it does not perform the optimization.

The resolution would appear to be to reverse the resolution of issue 2022 and guarantee that named return value optimization is **not** performed in constant expression evaluation.

**Notes from the March, 2018 meeting:**

CWG concurred with the suggested direction.

**Proposed resolution (November, 2018)**

1. Change 7.7 [expr.const] paragraph 1 as follows:

> Expressions that satisfy these requirements, assuming that copy elision **(10.9.5 [class.copy.elision])** is **not** performed, are called *constant expressions*. [Note: Constant expressions can be evaluated during translation. —end note]

2. Change 9.1.5 [dcl.constexpr] paragraph 7 as follows:

> A call to a constexpr function produces the same result as a call to an equivalent non-constexpr function in all respects except that
>
> - a call to a constexpr function can appear in a constant expression (7.7 [expr.const]) and
> - copy elision is ~~mandatory~~ **not performed** in a constant expression (10.9.5 [class.copy.elision]).

3. Change 10.9.5 [class.copy.elision] paragraph 1 as follows:

> ...Copy elision is ~~required~~ **not permitted** where an expression is evaluated in a context requiring a constant expression (7.7 [expr.const]) and in constant initialization (6.8.3.2 [basic.start.static]). [Note: Copy elision might ~~not~~ be performed if the same expression is evaluated in another context. —end note]

---

### 2303. Partial ordering and recursive variadic inheritance

**Section:** 12.9.2.1 [temp.deduct.call] **Status:** ready **Submitter:** John Spicer **Date:** 2016-05-24

The status of an example like the following is not clear:

```
template <typename... T>              struct A;
template <>                           struct A<> {};
template <typename T, typename... Ts> struct A<T, Ts...> : A<Ts...> {};
struct B : A<int> {};

template <typename... T>
void f(const A<T...>&);
```

```
void g() {
  f(B{});
}
```

This seems to be ambiguous in the current wording because `A<>` and `A<int>` both succeed in deduction. It would be reasonable to prefer the more derived specialization.

**Notes from the March, 2018 meeting:**

The relevant specification is in 12.9.2.1 [temp.deduct.call] bullet 4.3 and paragraph 5, which specifies that if there is more than one possible deduced `A`, deduction fails. The consensus was to add wording similar to that of overload resolution preferring "nearer" base classes.

**Proposed resolution (November, 2018):**

Change 12.9.2.1 [temp.deduct.call] bullet 4.3 as follows:

> In general, the deduction process attempts to find template argument values that will make the deduced A identical to A (after the type A is transformed as described above). However, there are three cases that allow a difference:
>
> - ...
> - If `P` is a class and `P` has the form *simple-template-id*, then the transformed `A` can be a derived class **`D`** of the deduced `A`. Likewise, if `P` is a pointer to a class of the form *simple-template-id*, the transformed `A` can be a pointer to a derived class **`D`** pointed to by the deduced `A`. **However, if there is a class `C` that is a (direct or indirect) base class of `D` and derived (directly or indirectly) from a class `B` and that would be a valid deduced `A`, the deduced `A` cannot be `B` or pointer to `B`, respectively. [*Example:***
>
> ```
> template <typename... T> struct X;
> template <> struct X<> {};
> template <typename T, typename... Ts> struct X<T, Ts...> : X<Ts...> {};
> struct D : X<int> {};
>
> template <typename... T>
> int f(const X<T...>&);
> int x = f(D());  // calls f<int>, not f<>
>                  // B is X<>, C is X<int>
> ```
>
> ***—end example*]**

---

### 2309. Restrictions on nested statements within `constexpr` functions

**Section:** 9.1.5 [dcl.constexpr] **Status:** ready **Submitter:** Faisal Vali **Date:** 2016-07-30

Section 9.1.5 [dcl.constexpr] bullet 3.4 specifies a list of constructs that that the body of a `constexpr` function shall not contain. However, the meaning of the word "contain" is not clear. For example, are things appearing in the body of a nested `constexpr` lambda "contained" in the body of the `constexpr` function?

**Proposed resolution (November, 2018):**

1. Add the following two paragraphs after 8 [stmt.stmt] paragraph 1:

   ...The optional *attribute-specifier-seq* appertains to the respective statement.

   **A *substatement* of a *statement* is one of the following:**

   - **for a *labeled-statement*, its contained *statement*,**
   - **for a *compound-statement*, any *statement* of its *statement-seq*,**
   - **for a *selection-statement*, any of its *statement*s (but not its *init-statement*), or**

- **for an *iteration-statement*, its contained *statement* (but not an *init-statement*).**

**[*Note:* The *compound-statement* of a *lambda-expression* is not a substatement of the *statement* (if any) in which the *lambda-expression* lexically appears. —*end note*]**

**A *statement* `S1` *encloses* a *statement* `S2` if**

- **`S2` is a substatement of `S1` (9 [dcl.dcl]),**
- **`S1` is a *selection-statement* or *iteration-statement* and `S2` is the *init-statement* of `S1`,**
- **`S1` is a *try-block* and `S2` is its *compound-statement* or any of the *compound-statement*s of its *handler*s, or**
- **`S1` encloses a statement `S3` and `S3` encloses `S2`.**

2. Delete the following sentence from 8.4 [stmt.select] paragraph 1:

~~In 8 [stmt.stmt], the term substatement refers to the contained statement or statements that appear in the syntax notation.~~

3. Change 9.1.5 [dcl.constexpr] bullet 3.3 as follows:

The definition of a constexpr function shall satisfy the following requirements:

- ...
- its *function-body* shall be `= delete`, `= default`, or a *compound-statement* that does not ~~contain~~ **enclose (8 [stmt.stmt]**
  - ...

---

**2310. Type completeness and derived-to-base pointer conversions**

**Section:** 7.3.11 [conv.ptr] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-08-08

The specification of derived-to-base pointer conversions in 7.3.11 [conv.ptr] paragraph 3 does not require that the derived class be complete at the point of the conversion. This leaves unclear the status of an example like the following, on which there is implementation divergence:

```
template<typename A, typename B> struct check_derived_from {
  static A a;
  static constexpr B *p = &a;
};
struct W {};
struct X {};
struct Y {};
struct Z : W,
  X, check_derived_from<Z, X>,  // #1
  check_derived_from<Z, Y>, Y { // #2
  check_derived_from<Z, W> cdf; // #3
};
```

**Notes from the March, 2018 meeting:**

The consensus of CWG was that the derived class must be complete at the point of the conversion, and thus all three attempted conversions in the example are ill-formed.

**Proposed resolution (November, 2018):**

1. Change 7.3.11 [conv.ptr] paragraph 3 as follows:

A prvalue of type “pointer to *cv* `D`”, where `D` is a **complete** class type, can be converted to a prvalue of type “pointer to *cv* `B`”, where `B` is a base class (10.6 [class.derived]) of `D`. If `B` is an inaccessible...

2. Change 7.3.12 [conv.mem] paragraph 2 as follows:

A prvalue of type “pointer to member of `B` of type *cv* `T`”, where B is a class type, can be converted to a prvalue of type “pointer to member of `D` of type *cv* `T`”, where `D` is a **complete class** derived ~~class~~ (10.6 [class.derived]) ~~of~~ **from** `B`. If `B` is an inaccessible...

3. Change 7.6.1.9 [expr.static.cast] paragraphs 11 and 12 as followed:

A prvalue of type “pointer to *cv1* `B`”, where `B` is a **complete** class type, can be converted to a prvalue of type “pointer to *cv2* `D`”, where D is a class derived (10.6 [class.derived]) from B , if cv2 is the same...

A prvalue of type “pointer to member of `D` of type *cv1* `T`” can be converted to a prvalue of type “pointer to member of `B` of type *cv2* `T`”, where `B` is a base class (10.6 [class.derived]) of **a complete class** `D`, if *cv2* is the same...

---

**2317. Self-referential default member initializers**

**Section:** 10.9.2 [class.base.init] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-08-29

Consider an example like:

```
struct A {
  int n = A{}.n;
};
```

There doesn't seem to be a good reason to support this kind of thing, and it would be simpler to say that a default member initializer can't trigger any direct or indirect use of itself in general, rather than just the two special cases that were banned by issues 1696 and 1397.

**Notes from the March, 2018 meeting:**

There was a suggestion that creating an object of the containing class in a default member initializer should be prohibited. That would presumably be a difference between the reference member and non-reference member cases, since the intent is to allow creation of a temporary for a reference member to bind to. The suggested approach for drafting was simply to remove the restriction to references in 9.3.1 [dcl.init.aggr] paragraph 11.

**Proposed resolution (November, 2018):**

Change 9.3.1 [dcl.init.aggr] paragraph 12 as follows:

If a ~~reference~~ member ~~is initialized from its~~ **has a** default member initializer and a potentially-evaluated subexpression thereof is an aggregate initialization that would use that default member initializer, the program is ill-formed. [*Example:*

```
struct A;
extern A a;
struct A {
  const A& a1 { A{a,a} }; // OK
  const A& a2 { A{} };    // error
};
A a{a,a};                 // OK

struct B {
int n = B{}.n;            // error
};
```

—*end example*]

---

**2318. Nondeduced contexts in deduction from a *braced-init-list***

**Section:** 12.9.2.5 [temp.deduct.type] **Status:** ready **Submitter:** Jonathan Caves **Date:** 2016-08-12

The status of an example like the following is unclear:

```
template<typename T, int N> void g(T (* const (&)[N])(T)) { }

int f1(int);
int f4(int);
char f4(char);

void f() {
  g({ &f1, &f4 });  // OK, T deduced to int, N deduced to 2?
}
```

The problem is the interpretation of 12.9.2.5 [temp.deduct.type] paragraph 4:

> In most cases, the types, templates, and non-type values that are used to compose `P` participate in template argument deduction. That is, they may be used to determine the value of a template argument, and the value so determined must be consistent with the values determined elsewhere. In certain contexts, however, the value does not participate in type deduction, but instead uses the values of template arguments that were either deduced elsewhere or explicitly specified. If a template parameter is used only in non-deduced contexts and is not explicitly specified, template argument deduction fails.

According to 12.9.2.1 [temp.deduct.call] paragraph 1, deduction is performed independently for each element of the initializer list:

> Template argument deduction is done by comparing each function template parameter type (call it `P`) that contains *template-parameter*s that participate in template argument deduction with the type of the corresponding argument of the call (call it `A`) as described below. If removing references and cv-qualifiers from `P` gives `std::initializer_list<P'>` or `P'[N]` for some `P'` and `N` and the argument is a non-empty initializer list (9.3.4 [dcl.init.list]), then deduction is performed instead for each element of the initializer list, taking `P'` as a function template parameter type and the initializer element as its argument, and in the `P'[N]` case, if `N` is a non-type template parameter, `N` is deduced from the length of the initializer list. Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (12.9.2.5 [temp.deduct.type]).

Deduction fails for the second element of the list, `&f4`, because of ambiguity. Does this mean that deduction fails for the entire call, or does the successful deduction of `T` from the first element and `N` from the length of the list result in successful deduction for the call?

**Notes from the July, 2017 meeting:**

CWG determined that the call is well-formed.

**Proposed resolution (November, 2018):**

Change 12.9.2.1 [temp.deduct.call] paragraph 1 as follows:

> Template argument deduction is done by comparing each function template parameter type (call it `P`) that contains *template-parameter*s that participate in template argument deduction with the type of the corresponding argument of the call (call it `A`) as described below. If removing references and cv-qualifiers from `P` gives `std::initializer_list<P'>` or `P' [N]` for some `P"` and `N` and the argument is a non-empty initializer list (9.3.4 [dcl.init.list]), then deduction is performed instead for each element of the initializer list **independently**, taking `P'` as ~~a~~ **separate** function template parameter type**s `P'`;** and the ***i*-th** initializer element as ~~its~~ **the corresponding** argument**.** ~~, and in~~ **In** the `P' [N]` case, if `N` is a non-type template parameter, `N` is deduced from the length of the initializer list. Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (12.9.2.5 [temp.deduct.type]). [*Example:*

```
...
template<class T, int N> void n(T const(&)[N], T);
n({{1},{2},{3}},Aggr()); // OK, T is Aggr, N is 3

template<typename T, int N> void o(T (* const (&)[N])(T)) { }
int f1(int);
int f4(int);
char f4(char);
o({ &f1, &f4 }); // OK, T deduced as int from first element, nothing deduced from second element, N deduced as 2
o({ &f1, static_cast<char(*)(char)>(&f4) }); // error: conflicting deductions for T
```

---

**2330. Missing references to variable templates**

**Section:** 12.8 [temp.spec] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2016-12-06

Presumably paragraphs 1-3 of 12.8 [temp.spec] are intended to apply to variable templates, but the term does not appear in the current wording of these paragraphs.

**Proposed resolution (November, 2018):**

1. Change 12.8 [temp.spec] paragraph 1 as follows:

> The act of instantiating a function, **a variable,** a class, a member of a class template or a member template is referred to as template instantiation.

2. Change 12.8 [temp.spec] paragraphs 3 and r as follows:

> An explicit specialization may be declared for a function template, **a variable template,** a class template, a member of a class template or a member template. An explicit specialization declaration is introduced by template<> . In an explicit specialization declaration for **a variable template,** a class template, a member of a class template or a class member template, the name of the **the variable or** class that is explicitly specialized shall be a *simple-template-id*. In the explicit specialization declaration for a function template or a member function template, the name of the function or member function explicitly specialized may be a *template-id*. [*Example:*...
>
> An instantiated template specialization can be either implicitly instantiated (12.8.1 [temp.inst]) for a given argument list or be explicitly instantiated (12.8.2 [temp.explicit]). A specialization is a class, **variable,** function, or class member that is either instantiated or explicitly specialized (12.8.3 [temp.expl.spec]).

---

**2331. Redundancy in description of class scope**

**Section:** 6.3.7 [basic.scope.class] **Status:** ready **Submitter:** Thomas Köppe **Date:** 2016-12-07

The first four paragraphs of 6.3.7 [basic.scope.class] are somewhat redundant. In particular:

- The normative paragraphs are 4 and 2.
- Paragraph 1 is subsumed by paragraph 4.
- Paragraph 3 follows from existing rules for name hiding.

This is editorial issue 1169.

**Proposed resolution (November, 2018):**

In 6.3.7 [basic.scope.class], delete paragraph 1, move paragraph 4 to the beginning, and make paragraph 3 a note:

~~The potential scope of a name declared in a class consists not only of the declarative region following the name's point of declaration, but also of all complete-class contexts (10.3 [class.mem]) of that class.~~

**The potential scope of a declaration that extends to or past the end of a class definition also extends to the regions defined by its member definitions, even if the members are defined lexically outside the class (this includes static data member definitions, nested class definitions, and member function definitions, including the member function body and any portion of the declarator part of such definitions which follows the *declarator-id*, including a *parameter-declaration-clause* and any default arguments (9.2.3.6 [dcl.fct.default])).**

A name `N` used in a class `S` shall refer to the same declaration in its context and when re-evaluated in the completed scope of `S`. No diagnostic is required for a violation of this rule.

**[*Note:*** A name declared within a member function hides a declaration of the same name whose scope extends to or past the end of the member function's class **(6.3.10 [basic.scope.hiding])**. ***—end note*]**

~~The potential scope of a declaration that extends to or past the end of a class definition also extends to the regions defined by its member definitions, even if the members are defined lexically outside the class (this includes static data member definitions, nested class definitions, and member function definitions, including the member function body and any portion of the declarator part of such definitions which follows the *declarator-id*, including a *parameter-declaration-clause* and any default arguments (9.2.3.6 [dcl.fct.default])).~~

---

**2332. *template-name* as *simple-type-name* vs injected-class-name**

**Section:** 9.1.7.2 [dcl.type.simple] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2016-12-10

Paper P0091R3 has the following example:

```
template<typename T> struct X {
  template<typename Iter>
  X(Iter b, Iter e) { /* ... */ }

  template<typename Iter>
  auto foo(Iter b, Iter e) {
    return X(b, e); // X<U> to avoid breaking change
  }

  template<typename Iter>
  auto bar(Iter b, Iter e) {
    return X<Iter::value_type>(b, e); // Must specify what we want
  }
};
```

The intent was presumably to avoid breaking existing code, but the new wording in 9.1.7.2 [dcl.type.simple] paragraph 2 appears to make the expression `X(b, e)` ill-formed:

> A *type-specifier* of the form `typename`$_{opt}$ *nested-name-specifier*$_{opt}$ *template-name* is a placeholder for a deduced class type (9.1.7.5 [dcl.type.class.deduct]). The *template-name* shall name a class template that is not an injected-class-name.

Suggested resolution:

Deleting the wording in question and replacing it with a cross-reference to 12.7.1 [temp.local], which makes it clear that the injected-class-name is a *type-name* and not a *template-name* in this context, would seem to address the problem adequately.

**Proposed resolution (November, 2018):**

Change 9.1.7.2 [dcl.type.simple] paragraph 2 as follows:

> The *simple-type-specifier* `auto` is a placeholder for a type to be deduced (9.1.7.4 [dcl.spec.auto]). A *type-specifier* of the form `typename`$_{opt}$ *nested-name-specifier*$_{opt}$ *template-name* is a placeholder for a deduced class type (9.1.7.5 [dcl.type.class.deduct]). The *template-name* shall name a class template ~~that is not an *injected-class-name*~~. **[*Note:***

**An injected-class-name is never interpreted as a *template-name* in contexts where a *type-specifier* may appear (12.7.1 [temp.local]). —*end note*]** The other *simple-type-specifier*s specify...

---

**2336. Destructor characteristics vs potentially-constructed subobjects**

**Section:** 13.4 [except.spec] **Status:** ready **Submitter:** Nathan Sidwell **Date:** 2017-02-28

According to 13.4 [except.spec] paragraph 8,

> The exception specification for an implicitly-declared destructor, or a destructor without a *noexcept-specifier*, is potentially-throwing if and only if any of the destructors for any of its potentially constructed subojects is potentially throwing.

10.3.3 [special] paragraph 5 defines “potentially constructed subobjects” as follows:

> For a class, its non-static data members, its non-virtual direct base classes, and, if the class is not abstract (10.6.3 [class.abstract]), its virtual base classes are called its potentially constructed subobjects.

This leads to the following problem:

```
class V {
public:
  virtual ~V() noexcept(false);
};

class B : virtual V {
  virtual void foo () = 0;
  // implicitly defined virtual ~B () noexcept(true);
};

class D : B {
  virtual void foo ();
  // implicitly defined virtual ~D () noexcept(false);
};
```

Here, `D::~D()` is throwing but overrides the non-throwing `B::~B()`.

There are similar problems with the deletedness of destructors per 10.3.6 [class.dtor] paragraph 5, which also only considers potentially constructed subobjects.

**Proposed resolution (November, 2018):**

Change 13.4 [except.spec] paragraph 8 as follows:

> The exception specification for an implicitly-declared destructor, or a destructor without a *noexcept-specifier*, is potentially-throwing if and only if any of the destructors for any of its potentially constructed subobjects is ~~potentially throwing~~ **potentially-throwing or the destructor is virtual and the destructor of any virtual base class is potentially-throwing**.

---

**2352. Similar types and reference binding**

**Section:** 9.3.3 [dcl.init.ref] **Status:** ready **Submitter:** Richard Smith **Date:** 2017-07-14

In an example like

```
int *ptr;
const int *const &f() {
```

```
  return ptr;
}
```

What is returned is a reference to a temporary instead of binding directly to `ptr`. The rules for reference-related types should say that `T` is reference-related to `U` if `U*` can be converted to `T*` by a qualification conversion.

**Notes from the April, 2018 teleconference:**

CWG agreed with the proposed direction.

**Proposed resolution (November, 2018):**

Change 9.3.3 [dcl.init.ref] paragraph 4 as follows:

> Given types "*cv1* `T1`" and "*cv2* `T2`", "*cv1* `T1`" is *reference-related* to "*cv2* `T2`" if `T1` is ~~the same type as~~ **similar (7.3.5 [conv.qual]) to** `T2`, or `T1` is a base class of `T2`. "*cv1* `T1`" is *reference-compatible* with "*cv2* `T2`" if
>
> - ~~`T1` is reference-related to `T2`, or~~
> - ~~`T2` is "`noexcept` function" and `T1` is "function", where the function types are otherwise the same,~~
>
> ~~and *cv1* is the same cv-qualification as, or greater cv-qualification than, *cv2*~~ **a prvalue of type "pointer to *cv2* `T2`" can be converted to the type "pointer to *cv1* `T1`" via a standard conversion sequence (7.3 [conv])**. In all cases where the ~~reference-related or~~ reference-compatible relationship of two types is used to establish the validity of a reference binding~~, and `T1` is a base class of `T2`~~ **and the standard conversion sequence would be ill-formed**, a program that necessitates such a binding is ill-formed ~~if `T1` is an inaccessible (10.8 [class.access]) or ambiguous (10.7 [class.member.lookup]) base class of `T2`~~.

---

**2358. Explicit capture of value**

**Section:** 7.5.5.2 [expr.prim.lambda.capture] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2017-09-20

The status of an example like the following is unclear:

```
int foo(int a = ([loc=1] { return loc; })()) { return a; }
```

because of 7.5.5.2 [expr.prim.lambda.capture] paragraph 9:

> A *lambda-expression* appearing in a default argument shall not implicitly or explicitly capture any entity.

However, there doesn't appear to be a good reason for prohibiting such a capture.

**Notes from the April, 2018 teleconference:**

CWG felt that the rule for capturing should be something like the prohibition for local classes odr-using a variable with automatic storage duration in 10.5 [class.local] paragraph 1.

**Proposed resolution (November, 2018):**

Change 7.5.5.2 [expr.prim.lambda.capture] paragraph 9 as follows:

> A *lambda-expression* appearing in a default argument shall not implicitly or explicitly capture any entity**, except for an *init-capture* for which any full-expression in its *initializer* satisfies the constraints of an expression appearing in a default argument (9.2.3.6 [dcl.fct.default])**. [*Example:*
>
> ```
> void f2() {
>   int i = 1;
>   void g1(int = ([i]{ return i; })());       // ill-formed
>   void g2(int = ([i]{ return 0; })());       // ill-formed
>   void g3(int = ([=]{ return i; })());       // ill-formed
>   void g4(int = ([=]{ return 0; })());       // OK
> ```

```
    void g5(int = ([]{ return sizeof i; })()); // OK
    void g6(int = ([x=1] { return x; })();     // OK
    void g7(int = ([x=i] { return x; })();     // ill-formed
  }
```

—*end example*]

---

**2360. `[[maybe_unused]]` and structured bindings**

**Section:** 9.11.8 [dcl.attr.unused] **Status:** ready **Submitter:** Michael Wong **Date:** 2017-10-19

According to9.11.8 [dcl.attr.unused] paragraph 2,

> The attribute may be applied to the declaration of a class, a *typedef-name*, a variable, a non-static data member, a function, an enumeration, or an enumerator.

This does not include structured bindings, although there seems to be no good reason to prohibit uses like

```
[[maybe_unused]] auto [a, b] = std::make_pair(42, 0.23);
```

**Notes from the October, 2018 teleconference:**

CWG agreed that such an annotation should be permitted and apply to the underlying variable; i.e., a compiler might warn in the absence of such an attribute if none of the structured bindings were used, and the presence of the attribute would silence such warnings.

**Proposed resolution (November, 2018):**

Change 9.11.8 [dcl.attr.unused] paragraphs 2 and 3 as follows:

> The attribute may be applied to the declaration of a class, a *typedef-name*, a variable **(including a structured binding declaration)**, a non-static data member, a function, an enumeration, or an enumerator.
>
> ~~[*Note:*~~ For an entity marked `maybe_unused`, implementations should not emit a warning that the entity ~~is~~ **or its structured bindings (if any) are used or** unused**.** ~~, or that the entity is used despite the presence of the attribute. —*end note*]~~ **For a structured binding declaration not marked `maybe_unused`, implementations should not emit such a warning unless all of its structured bindings are unused.**